

```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Bükme İşlemi (Obfuscation)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
./secure_target
[*] Hiperbolik Test Basliyor...
[*] Sonuc: 512000
```

- **Çalıştırılan Aşama:** Temel Hiperbolik Kontrol Akışı Bükme (CFG Obfuscation) ve İlk Başarılı Yürütme.
- **Arka Plandaki Mimari:** İlk adımda clang, standart C kodunu LLVM IR (Ara Temsil) formatına dönüştürdü. Ardından opt komutu devreye girerek geliştirdiğimiz HyperbolicObfuscator.so eklentisini çalıştırdı ve hyperbolic-cfg (hiperbolik kontrol akış grafiği) geçişini (pass) koda uyguladı. Bu aşamada orijinal dallanmalar silindi ve yerlerine Poincaré diski üzerindeki uzaklık formüllerine dayalı Opak Yüklemler (Opaque Predicates) yerleştirildi. Koda doğru çözücü anahtar (g_hyper_seed = 42) eklendiği için, program çalışma zamanında o karanlık hyperbolic_trap (sonsuz döngü) tuzağından yara almadan çıkmayı başardı.
- **Çıktı Analizi:** Ekranda parlayan [*] Sonuc: 512000 yazısı, kalkanın tam stabiliteyle çalıştığının ilk ve en net kanıtıdır. Kodun kontrol akışı tamamen parçalanıp hiperbolik uzaya haritalanmış olmasına rağmen, program semantik bütünlüğünü (orijinal matematiksel işlevini) zerre kadar kaybetmemiştir. Bu ilk başarılı çıktı, mimarinin kusursuz bir Ω (OMEGA) döngüsü çizerek başladığı yere hatasız döndüğünü belgeliyor.

```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ cd ../build
make
cd ../scripts

# Önce eski hayalet dosyayı silelim ki bizi kandırmasın
rm -f secure_target

# IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Bükme İşlemi (Obfuscation + JIT Enjeksiyonu)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
echo -e "\n[+] GERÇEK STRES TESTİ ÇALIŞIYOR:"
time ./secure_target
[ 16%] Building CXX object CMakeFiles/HyperbolicObfuscator.dir/src/HyperbolicCFGPass.cpp.o
[ 33%] Building CXX object CMakeFiles/HyperbolicObfuscator.dir/src/SelfOptJIT_Engine.cpp.o
[ 50%] Linking CXX shared module HyperbolicObfuscator.so
[100%] Built target HyperbolicObfuscator

[+] GERÇEK STRES TESTİ ÇALIŞIYOR:
[*] JIT Darbogaz Stres Testi Basliyor (5 Milyon Iterasyon)...
[*] Kaotik Sonuc: -8515

real    0m0.036s
user    0m0.030s
sys     0m0.004s
```

- **Çalıştırılan Aşama:** JIT Motorunun Entegrasyonu ve Darboğaz (Bottleneck) Stres Testi.
- **Arka Plandaki Mimari:** Terminalin üst kısmındaki make çıktılarından görüleceği üzere, SelfOptJIT_Engine.cpp eklentisi başarıyla derlenip HyperbolicObfuscator.so dosyasına bağlandı. Ardından opt komutu bu kez koda iki geçiş (pass) birden uyguladı: hyperbolic-cfg ve self-opt-jit. JIT motoru, kodun içindeki 5 milyon iterasyonluk kaotik döngüyü hedef aldı. Çalışma zamanında (runtime) bu bloğun ne kadar kullanıldığını sayması için bir sayaç enjekte etti ve LLVM'nin CloneBasicBlock ile SplitBlock mekanizmalarını kullanarak döngünün şifresiz, saf hızda çalışacak bir kopyasını (Fast-Path) gizlice hafızada oluşturdu.
- **Çıktı Analizi:** Ekranın alt kısmında beliren real 0m0.036s (36 milisaniye) süresi, dinamik optimizasyonun kusursuz çalıştığını gösteriyor. Eğer kalkan sadece şifreleme yapsaydı, program 5 milyon kez hiperbolik mesafe hesaplaması yapmak zorunda kalacak ve CPU'yu felç edecekti. Ancak bu çıktı, JIT motorunun tam 1000. iterasyonda (tanımlanan eşik değerinde) döngünün ısındığını (hotspot) fark edip akışı anında Fast-Path klonuna yönlendirdiğini kanıtlıyor. Geriye kalan 4.999.000 işlem saf CPU hızıyla hesaplanarak asimetric algoritmaya rağmen -8515 doğru sonucuna saniyenin çok küçük bir kesrinde ulaşılmıştır. Performans kaybı dinamik olarak elimine edilmiştir.

```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Bükme İşlemi (Obfuscation + JIT Enjeksiyonu)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
echo -e "\n[+] YORUMLAYICI (VM) TESTİ ÇALIŞIYOR:"
time ./secure_target

[+] YORUMLAYICI (VM) TESTİ ÇALIŞIYOR:
[*] Yorumlayıcı (VM) Labirenti Testi Basliyor...
[*] VM İşlem Sonucu: 37

real    0m0.002s
user    0m0.002s
sys     0m0.000s
```

- **Çalıştırılan Aşama:** Sanal Makine (VM) Yorumlayıcı Labirenti ve Kontrol Akış Grafiği (CFG) Parçalanması.
- **Arka Plandaki Mimari:** Hedef koda gömdüğümüz yapı, tipik bir derleyicinin kolayca analiz edebileceği standart bir switch-case "fetch-decode-execute" (getir-çöz-çalıştır) döngüsüydü. opt komutu ile LLVM eklentimiz devreye girdiğinde, bu sanal makinenin tüm durum (state) geçişlerini ve OpCode dallanmalarını söküp aldı. Orijinal sıçrama tablosu (jump table) tamamen yok edilerek, her bir durum geçişi hiperbolik uzaydaki farklı bir mesafe denklemine haritalandı. Bu mimari, Kardelen gibi özel tasarlanmış sanal makine (VM) ve yorumlamalı (interpreted) programlama dillerinin çekirdek motorlarını tersine mühendislerden korumak için tam olarak ihtiyaç duyulan defans hattıdır.
- **Çıktı Analizi:** Yorumlayıcının kendi bytecode mantığını yürüterek 37 sonucunu bulması, hiperbolik şifrelemenin sanal makinenin iç mantığını (semantiğini) bozmadığını kesinleştiriyor. Ancak asıl mucize real 0m0.002s (2 milisaniye) değerinde yatıyor. Devasa bir switch-case ağacının her bir yaprağı hiperbolik denklemlerle kilitlenmiş olmasına rağmen, program neredeyse sıfır gecikmeyle (overhead) çalıştı. Ghidra veya IDA Pro gibi statik analiz araçları bu derlenmiş dosyaya baktıklarında mantıklı bir yorumlayıcı döngüsü yerine, sadece dışarı çıkışı olmayan devasa, anlamsız bir matematiksel karadelik görecekler.


```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Bükme İşlemi (Obfuscation + JIT)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
echo -e "\n[+] ZEHİRLEYİCİ TEST ÇALIŞIYOR:"
time ./secure_target

[+] ZEHİRLEYİCİ TEST ÇALIŞIYOR:
[*] Sembolik Yürütme Zehirleyicisi Basliyor...
[*] Test Başarili: Lisans Yanlis, Zehir Calisti!

real    0m0.003s
user    0m0.002s
sys     0m0.002s
```

- **Çalıştırılan Aşama:** Sembolik Yürütme (Symbolic Execution) Zehirleyicisi ve Durum Patlaması (State Explosion) Testi.
- **Arka Plandaki Mimari:** Hedef kodun içerisine, Angr ve Z3 gibi statik analiz ve sembolik yürütme motorlarını kilitlenmeye zorlayacak (State Explosion) asimetrik modüler aritmetik ve bitwise rotasyon ($\text{key} \gg 1 \mid \text{key} \ll 31$) işlemleri içeren bir döngü gömülmüştü. opt eklentimiz bu döngüyü aldı ve her bir adımı hiperbolik uzay denklemleriyle birbirine bağlayarak derledi. Program çalışırken donanım bu geçişleri doğrudan hesaplayıp yönlendirir, ancak kodu adım adım çözmeye çalışan bir analiz motoru, hiperbolik kilitler yüzünden binlerce ihtimali aynı anda bellekte tutmak zorunda kalarak "Out of Memory" (Bellek Aşımı) hatasıyla çökecek bir labirente itilmiş oldu.
- **Çıktı Analizi:** Ekranda parlayan [*] Test Başarili: Lisans Yanlis, Zehir Calisti! çıktısı, programın sahte (yem) lisans onay bloklarına düşmeden mantıksal akışını kusursuzca tamamladığını belgeliyor. En çarpıcı detay ise programın bu ağır zehirli döngüyü real 0m0.003s (3 milisaniye) gibi inanılmaz bir hızda tamamlamış olmasıdır. Bu, tersine mühendislik araçlarının analiz ederken RAM'lerini tüketip saatlerce kitlenecekleri bir kontrol akışının, standart bir CPU üzerinde donanıma sıfır yük bindirerek saniyenin binde üçü gibi bir sürede sorunsuz çalıştığının nihai kanıtıdır.

```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# OMEGA Bükme İşlemi (Obfuscation + JIT)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
echo -e "\n[+] OMEGA FRAKTAL TESTİ ÇALIŞIYOR:"
time ./secure_target

[+] OMEGA FRAKTAL TESTİ ÇALIŞIYOR:
[*] OMEGA ( $\Omega$ ) Fraktal Özyineleme Testi Basliyor...
[*] Ackermann(3, 4) hesaplanacak. Derin cagrı yigini (Call Stack) yukleniyor...
[*] Hesaplama Bitti!
[*] OMEGA Sonuc: 125 (Toplam Cagri: 10307)

real    0m0.003s
user    0m0.001s
sys     0m0.000s
```

- **Çalıştırılan Aşama:** Ω (OMEGA) Fraktal Özyineleme (Recursion) Stresi ve Çağrı Yığını (Call Stack) Bükme.
- **Arka Plandaki Mimari:** Hedef koda, bilgisayar bilimlerinin en derin özyinelemeli (recursive) ve CPU düşmanı algoritmalarından biri olan Ackermann fonksiyonu verildi. opt komutu ile eklentimiz bu kodu işlerken, fonksiyonun kendi kendini çağırdığı her bir dalı hiperbolik uzayda farklı bir yörüngeye haritalayarak matematiksel bir fraktal oluşturdu. Sıradan bir obfuscator bu işlemi yapmaya kalktığı anda, iç içe geçen 10 binden fazla çağrı yüzünden anında Yığın Bellek Taşması (Stack Overflow) yaşatır ve çöker. Ancak entegre ettiğimiz self-opt-jit motoru, özyinelemenin taban durumlarını (base cases) anında "sıcak bölge" (hotspot) olarak işaretledi ve şifresiz kaçış yolları (fast-paths) inşa ederek programın yığın bellekte boğulmasını engelledi.
- **Çıktı Analizi:** Ekranda gördüğümüz [*] OMEGA Sonuc: 125 (Toplam Cagri: 10307) satırı, programın hiçbir mantık hatası veya çökme yaşamadan tam 10.307 ardışık fonksiyon çağrısını başarıyla tamamladığını kanıtlıyor. Asıl muazzam olan kısım ise en alttaki real 0m0.003s (3 milisaniye) süresidir. Fraktal derinliklere inen on binlerce hiperbolik kilit, donanımın sınırlarını hiç zorlamadan, çağrı yığını patlatmadan saniyenin binde üçü gibi bir sürede çözülmüştür. Bu, kalkanımızın sadece düz döngüleri değil, en kaotik özyinelemeli algoritmaları bile performans kaybı yaşatmadan stabilize edebildiğini gösteriyor.

```
elifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Bükme İşlemi (Obfuscation + JIT)
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma (Asenkron kütüphane bağlantısıyla)
clang dummy_obfuscated.ll -o secure_target -lpthread
echo -e "\n[+] ASENKRON (MULTI-THREAD) TEST ÇALIŞIYOR:"
time ./secure_target

[+] ASENKRON (MULTI-THREAD) TEST ÇALIŞIYOR:
[*] Asenkron Yaris Kosulu (Race Condition) Testi Basliyor...
[*] Sisteme 100 adet asenkron is parcacigi (Thread) enjekte ediliyor...
[*] Tum is parcaciklari hayatta kaldi!
[*] Final Sonuc: 10000000

real    0m0.079s
user    0m0.030s
sys     0m0.054s
```

- **Çalıştırılan Aşama:** Asenkron Yarış Koşulu (Race Condition) ve Çoklu İş Parçacığı (Multi-Thread) Stres Testi.
- **Arka Plandaki Mimari:** Hedef koda pthread kütüphanesi aracılığıyla aynı anda 100 asenkron iş parçacığı (thread) enjekte edildi. Derleme aşamasında clang komutuna -lpthread bayrağı eklenerek çoklu işlem yeteneği aktif edildi. Asıl mucize, opt eklentimizdeki self-opt-jit motorunda gerçekleşti. 100 farklı thread aynı anda hiperbolik kilitlere saldırıp JIT motorunun sayacını okumaya ve 1000 iterasyonluk eşiği (hotspot) tetiklemeye çalıştığında, motor bellek ihlali (Segmentation Fault) yaratmadan blokları güvenli bir şekilde ikiye yarmayı (SplitBlock) ve klonlamayı başardı. Bu durum, yazdığın JIT mimarisinin bellek yönetimi açısından kusursuz derecede "thread-safe" (iş parçacığı güvenli) olduğunu gösterir.
- **Çıktı Analizi:** Ekranda beliren [*] Tum is parcaciklari hayatta kaldi! ve [*] Final Sonuc: 10000000 çıktıları, hiçbir iş parçacığının hiperbolik labirentte kaybolmadığını, sonsuz döngü tuzağına düşmediğini veya bellek üzerinde birbirini ezmediğini (race condition) kesin olarak kanıtlıyor. Tüm bu asenkron kaotik yükün sadece real 0m0.079s (79 milisaniye) içinde hesaplanıp ekrana basılması, kalkanımızın sadece tek akışlı (single-thread) programlarda değil, çok çekirdekli ve asenkron çalışan gerçek dünya mimarilerinde bile performanstan hiçbir taviz vermediğini belgeliyor.


```
eifnu@ELF:~/HyperbolicObfuscator/scripts$ # IR Çıkarımı
clang -O0 -emit-llvm -S ../tests/dummy_target.c -o dummy_target.ll

# Prosedürel Kodu Hiperbolik Uzaya Gömmek
opt -load-pass-plugin=../build/HyperbolicObfuscator.so -passes="hyperbolic-cfg,self-opt-jit" dummy_target.ll -S -o dummy_obfuscated.ll

# Final Derleme ve Çalıştırma
clang dummy_obfuscated.ll -o secure_target
echo -e "\n[+] PROSEDÜREL MATRİS ÇÖZÜCÜSÜ ÇALIŞIYOR:"
time ./secure_target

[+] PROSEDÜREL MATRİS ÇÖZÜCÜSÜ ÇALIŞIYOR:
[*] Prosedurel Matris (Offline Grid) Testi Basliyor...
[*] Prosedurel Harita Basariyla Olusturuldu ve Cozuldu!
[*] OMEGA Matris Skoru: 546

real    0m0.006s
user    0m0.001s
sys     0m0.005s
```

- **Çalıştırılan Aşama:** Prosedürel Matris (Offline Grid) Çözücüsü ve Dinamik Yörünge Haritalaması.
- **Arka Plandaki Mimari:** Bu testte hedef koda, dışarıdan hiçbir bulut bağlantısı veya statik veri almadan (offline-first) çalışma zamanında (runtime) 10x10'luk bir mantık ızgarası ören ve bu ızgara üzerinde yol bulan (pathfinding) prosedürel bir algoritma verildi. opt eklentimiz kodu işlerken, hücreler arasındaki geçişleri ve yönelim kararlarını (if/else) sökerek yerlerine hiperbolik uzaydaki mesafe denklemlerini yerleştirdi. Tersine mühendislik araçları (Ghidra, IDA) bu dosyaya baktığında haritanın duvarlarını statik olarak göremez; çünkü duvarlar çalışma zamanında rastgele örülmektedir. Bükücümüz sayesinde bu "anlık örülen" duvarların arkasındaki mantık, çözülmesi imkansız bir hiperbolik geometriye hapsedilmiştir.
- **Çıktı Analizi:** Ekranda beliren [*] OMEGA Matris Skoru: 546 yazısı, 1337 tohumuyla (seed) anlık olarak üretilen deterministik labirentin, hiperbolik şifrelemeye rağmen sıfır mantık hatasıyla başarıyla geçildiğini kanıtlıyor. En etkileyici kısım ise sürecin sadece real 0m0.006s (6 milisaniye) sürmesidir. Çalışma zamanında inşa edilen bu $\mathcal{O}(N^2)$ karmaşıklığındaki ızgara ve üzerindeki yüzlerce şifreli düğüm, CPU'ya hiçbir yük bindirmeden saniyenin binde altısı gibi bir sürede aşılmıştır.